

Constraint satisfaction problem

1. Variables:

A set of N variables $X_1, \dots, X_n$ that each take on a single value from some defined set of values.

2. Domain:

A set $\{x_1, \dots, x_n\}$ representing all possible values that a CSP variable can take on.

3. Constraints:

Define restrictions on the values of variables, potentially with regard to other variables.

Examples,

• Map coloring

1. Variables: WA, NT, Q, NSW, V, SA, T

2. Domains: $\{\text{red, green, blue}\}$

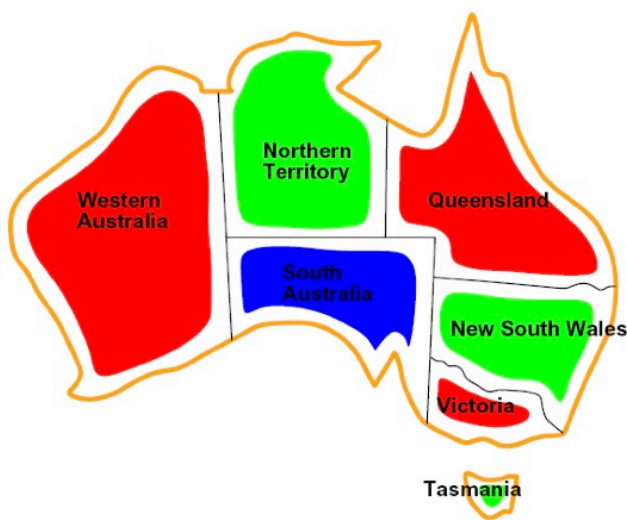
3. Constraints: Adjacent regions must have different colors

$WA \neq NT$ (Implicit)

$(WA, NT) \in \{(\text{red, green}), (\text{red, blue}), \dots\}$

Solutions are assignments satisfying all constraints:

$\{WA = \text{red}, NT = \text{green}, Q = \text{red}, NSW = \text{green}, V = \text{red}, SA = \text{blue}, T = \text{green}\}$



Constraint Graphs

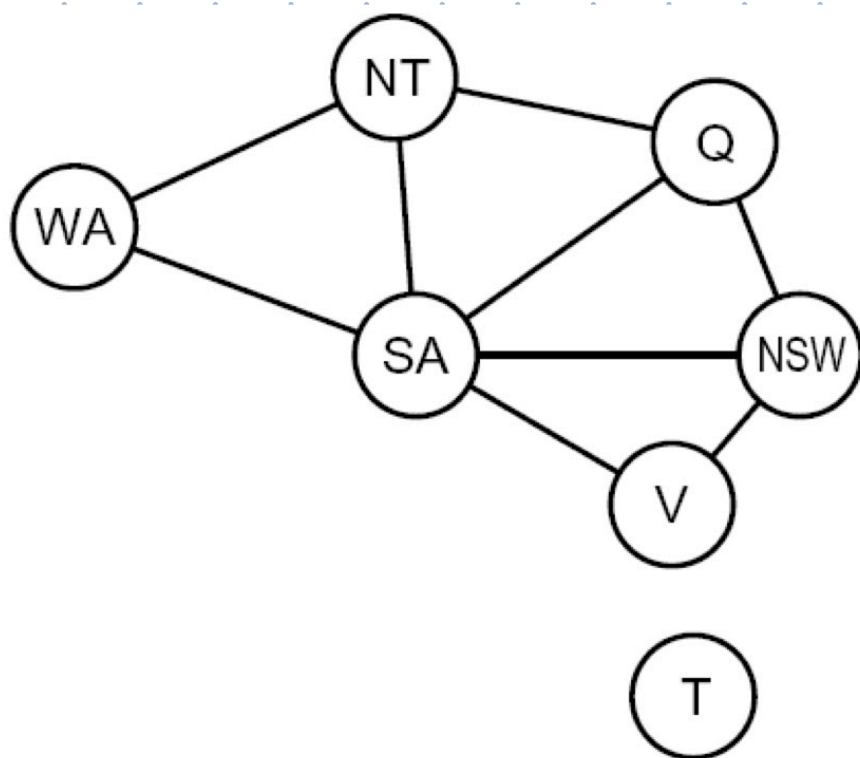
Binary CSPs:

Each constraint relates (at most) two variables

Binary constraint graph:

Nodes are variables, edges show constraints

General purpose CSP use graph structure to speed up search



Example:

1. N-Queens:

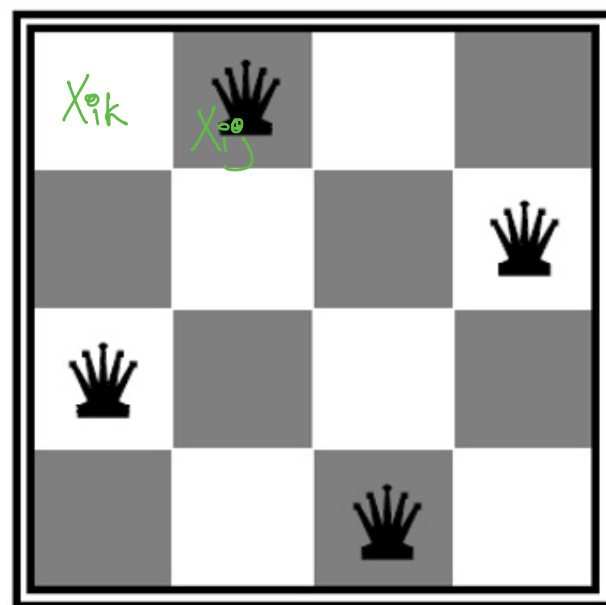
Formulation of: $\rightarrow$ Total variables $N \times N$

• Variables: X_{ij}

• Domains: $\{0, 1\}$

• Constraints

0 - Absent
1 - Present



$\forall i, j, k (X_{ij}, X_{ik}) \in \{(0,0), (0,1), (1,0)\}$ $\rightarrow$ compare 2 variables at a time.

$\forall i, j, k (X_{ij}, X_{kj}) \in \{(0,0), (0,1), (1,0)\}$

$\forall i, j, k (X_{ij}, X_{i+k, j+k}) \in \{(0,0), (0,1), (1,0)\}$

$\forall i, j, k (X_{ij}, X_{i+k, j-k}) \in \{(0,0), (0,1), (1,0)\}$

Formulation 2: $\rightarrow$ Total variables N

• Variables: Q_k

• Domains: $\{1, 2, 3, \dots, N\}$

• Constraints

$$(Q_1, Q_2) \in \{(1, 3), (1, 4), \dots\}$$

$\begin{matrix} \text{row_idx} & & \text{col_idx} \\ \text{row_idx} & & \text{col_idx} \end{matrix}$

	1	2	3	4
Q ₁		👑		
Q ₂				👑
Q ₃	👑			
Q ₄			👑	

2. Crypto-arithmetic

• Variables: $F, T, U, W, R, O, X_1, X_2, X_3$

• Domains: $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$

• Constraints

$$\text{alldiff}(F, T, U, W, R, O)$$

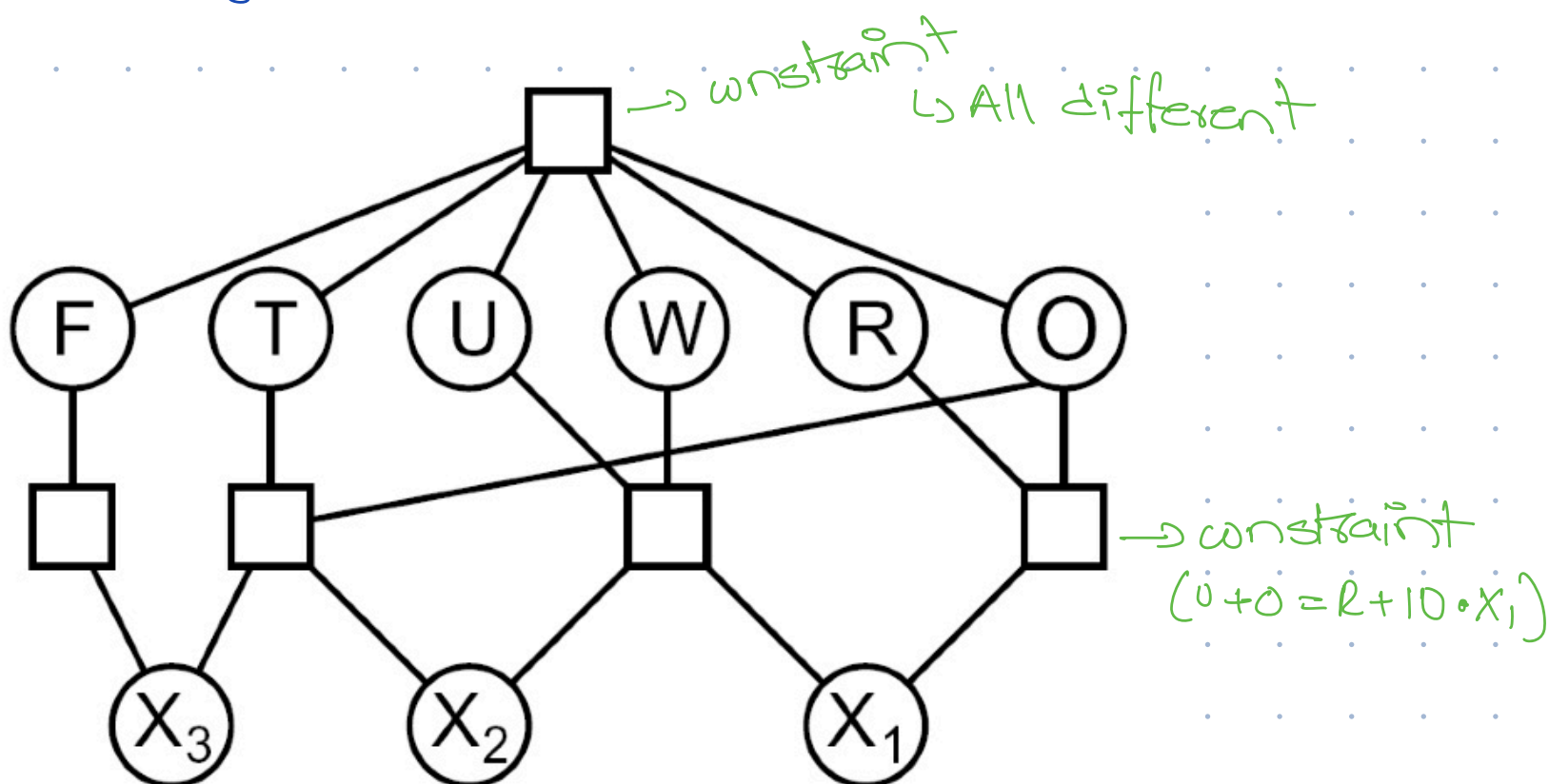
$$O + O = R + 10 \cdot X_1$$

$$X_1 + W + W = U + 10 \cdot X_2$$

$$X_2 + T + T = O + 10 \cdot X_3$$

$$F = X_3$$

$$\begin{array}{r} X_2 X_1 \\ T W O \\ + T W O \\ \hline F O U R \\ X_3 \\ 8 \\ 8 \\ \hline 16 \\ 6 + 10 \times 1 \end{array}$$



3. Sudoku

- Variables: $A_{1..9}, B_{1..9}, \dots$
- Domains: $\{1, 2, 3, \dots, 9\}$
- Constraints

$Alldiff(A1, A2, A3, A4, A5, A6, A7, A8, A9)$

$Alldiff(B1, B2, B3, B4, B5, B6, B7, B8, B9)$

...

$Alldiff(A1, B1, C1, D1, E1, F1, G1, H1, I1)$

$Alldiff(A2, B2, C2, D2, E2, F2, G2, H2, I2)$

...

$Alldiff(A1, A2, A3, B1, B2, B3, C1, C2, C3)$

$Alldiff(A4, A5, A6, B4, B5, B6, C4, C5, C6)$

...

	1	2	3	4	5	6	7	8	9
A			3		2		6		
B	9			3		5			1
C			1	8		6	4		
D			8	1		2	9		
E	7								8
F			6	7		8	2		
G			2	6		9	5		
H	8			2		3			9
I			5		1		3		

Constraints

Unary: $SA \neq \text{green} \rightarrow$ involves 1 CSP variable

Binary: $SA \neq WA \rightarrow$ involves 2 CSP variables

N-ary: use $\square$ (hypernode) $\rightarrow$ involves N CSP variables

↳ CSP with finite domain can be solved efficiently

Time complexity: $\Theta(d^n)$

where,

d = size of domain

n = no. of variables

Backtracking Search:

1. One variable at a time

↳ Variable assignments are commutative

2. Check constraints as you go.

↳ Consider values that do not conflict previous assignments.

↳ DFS with above 2 points is called backtracking.

Algorithm:

```
function BACKTRACKING-SEARCH(csp) returns solution/failure
  return RECURSIVE-BACKTRACKING({ }, csp)

function RECURSIVE-BACKTRACKING(assignment, csp) returns soln/failure
  if assignment is complete then return assignment
  var ← SELECT-UNASSIGNED-VARIABLE(VARIABLES[csp], assignment, csp)
  for each value in ORDER-DOMAIN-VALUES(var, assignment, csp) do
    if value is consistent with assignment given CONSTRAINTS[csp] then
      add {var = value} to assignment
      result ← RECURSIVE-BACKTRACKING(assignment, csp)
      if result ≠ failure then return result
      remove {var = value} from assignment
  return failure
```

Improving backtracking

Filtering: → 1st improvement

↳ Keep track of domains for unassigned variables & cross off bad ones.

↳ One such filtering technique is:

- Constraint propagation:

↳ Reason from constraint to constraint

↳ Two type of techniques:

1. Forward checking
2. Arc consistency algorithm

Consistency of an arc:

An arc $X \rightarrow Y$ is consistent iff each value in X 's domain must have a corresponding value in Y 's domain.

Remove values in the domain of x if there isn't a corresponding legal y .

Example,

Variables: X, Y

Constraint: $X \neq Y$

Domain: $\{1, 2, 3\}$

if $X=1$, then Y still have $\{2, 3\}$ that satisfies the constraint.

Forward checking:

Cross off values that violate a constraint when added to existing assignment

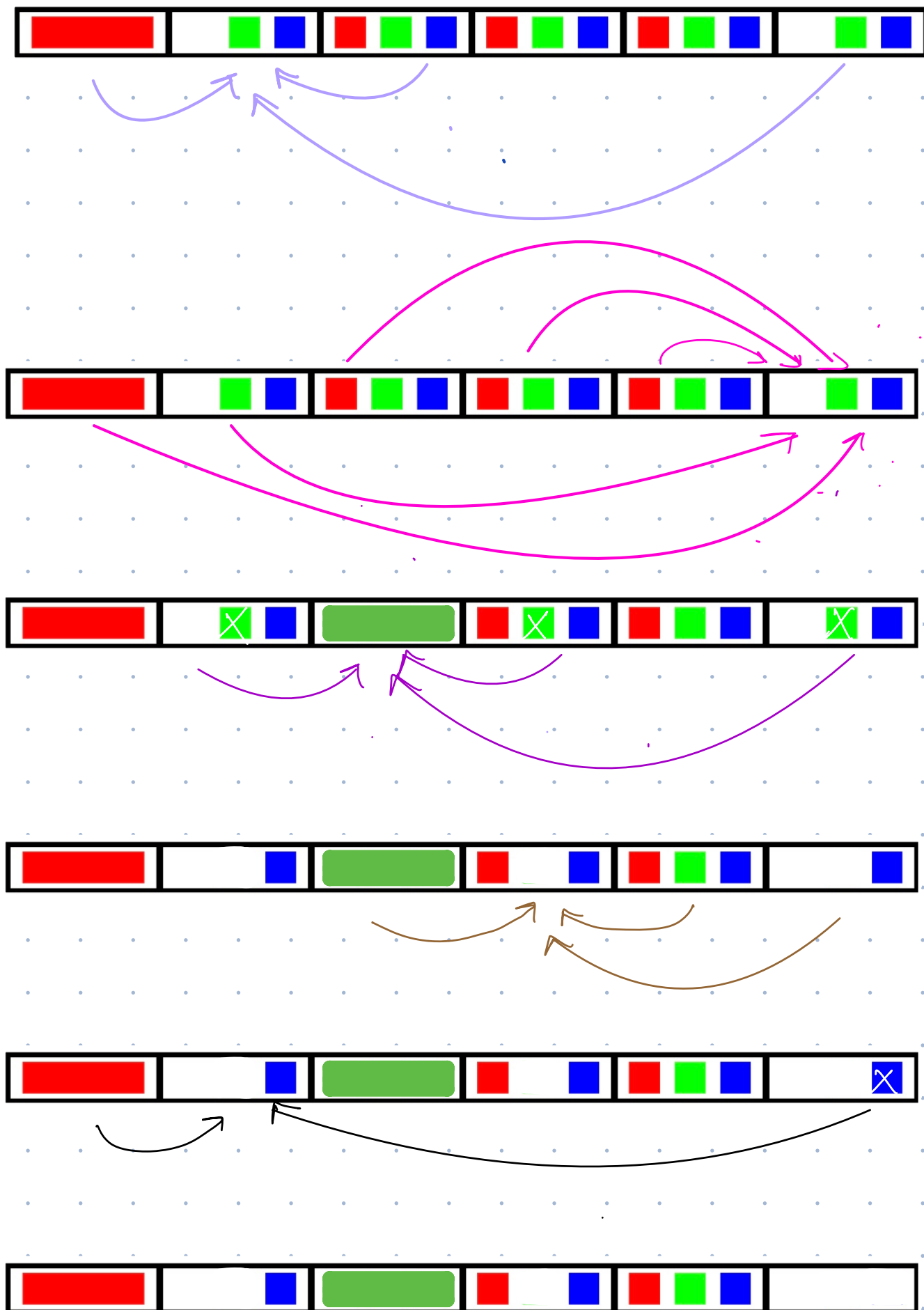


This approach checks arc consistency for direct neighbours of the assigned node but it is not that good

This wasn't detected.

Arc consistency algorithm (AC-3)





Queue
~~(NT, WA)~~
~~(SA, WA)~~

~~(WA, NT)~~
~~(Q, NT)~~
~~(SA, NT)~~

~~(WA, SA)~~
~~(NT, SA)~~
~~(Q, SA)~~
~~(NSW, SA)~~
~~(V, SA)~~

~~(NSW, Q)~~
~~(NT, Q)~~
~~(SA, Q)~~

~~(Q, NSW)~~
~~(V, NSW)~~
~~(SA, NSW)~~

~~(WA, NT)~~
~~(SA, NT)~~
~~(Q, NT)~~

~~(WA, SA)~~
~~(NT, SA)~~
~~(NSW, SA)~~
~~(V, SA)~~

no need
to check

As SA has no valid assignment left, we now know in just two assignments that WA=red & Q=green will not yield a valid solution

function AC-3(*csp*) returns the CSP, possibly with reduced domains

inputs: *csp*, a binary CSP with variables $\{X_1, X_2, \dots, X_n\}$

local variables: *queue*, a queue of arcs, initially all the arcs in *csp*

while *queue* is not empty **do**

$(X_i, X_j) \leftarrow \text{REMOVE-FIRST}(\text{queue})$

if REMOVE-INCONSISTENT-VALUES(X_i, X_j) **then**

for each X_k **in** NEIGHBORS[X_i] **do**

 add (X_k, X_i) to *queue*

function REMOVE-INCONSISTENT-VALUES(X_i, X_j) **returns** true iff succeeds

removed $\leftarrow$ false

for each x **in** DOMAIN[X_i] **do**

if no value y in DOMAIN[X_j] allows (x, y) to satisfy the constraint $X_i \leftrightarrow X_j$

then delete x from DOMAIN[X_i]; *removed* $\leftarrow$ true

return *removed*

Ordering $\rightarrow$ 2nd improvement

How to choose our variable to assign in backtracking?
before we selected randomly or first un-assigned but both of these are sub-optimal.

$var \leftarrow \text{SELECT-UNASSIGNED-VARIABLE}(\text{VARIABLES}[csp], \text{assignment}, csp)$

Instead,

1. Minimum Remaining Value / most constraint variable / fail-first / fail-fast

Choose variable with fewest legal left values in its domain.

2. Least constraint Value

Choose variable that rules out the fewest remaining values in the remaining variables.

Structure $\rightarrow$ 3rd improvement

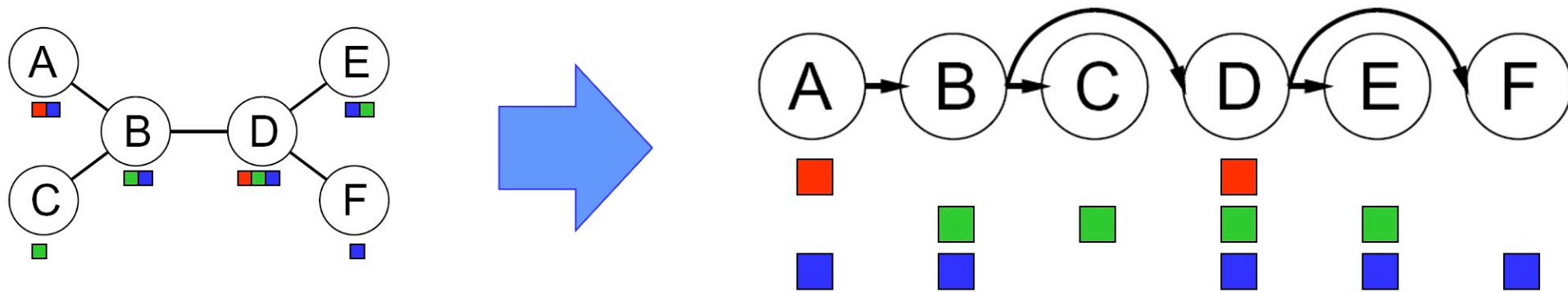
To convert constraint graph to tree, graph needs to be acyclic

First, pick an arbitrary node in the constraint graph for the CSP to serve as the root of the tree.

If there is, consider them independent problem (Any solution for mainland + solution for T is a solution for whole map)

Convert all undirected edges in the graph to directed edges pointing away from the root node (designated as node A).

Linearize (or topologically sort) the resulting directed acyclic graph by ordering the nodes such that parents precede children



Remove backward:

For $i = n : 2$, apply $\text{RemoveInconsistent}(\text{Parent}(X_i), X_i)$

Assign forward:

For $i = 1 : n$, assign X_i consistently with $\text{parent}(X_i)$

function TREE-CSP-SOLVER(csp) **returns** a solution, or *failure*

inputs: csp , a CSP with components X , D , C

$n \leftarrow$ number of variables in X

$\text{assignment} \leftarrow$ an empty assignment

$\text{root} \leftarrow$ any variable in X

$X \leftarrow \text{TOPOLOGICALSORT}(X, \text{root})$

for $j = n$ **down to** 2 **do**

 MAKE-ARC-CONSISTENT(PARENT(X_j), X_j)

if it cannot be made consistent **then return** *failure*

for $i = 1$ **to** n **do**

$\text{assignment}[X_i] \leftarrow$ any consistent value from D_i

if there is no consistent value **then return** *failure*

return assignment

The TREE-CSP-SOLVER algorithm for solving tree-structured CSPs. If the CSP has a solution, we will find it in linear time; if not, we will detect a contradiction.

What if graph has a loop?

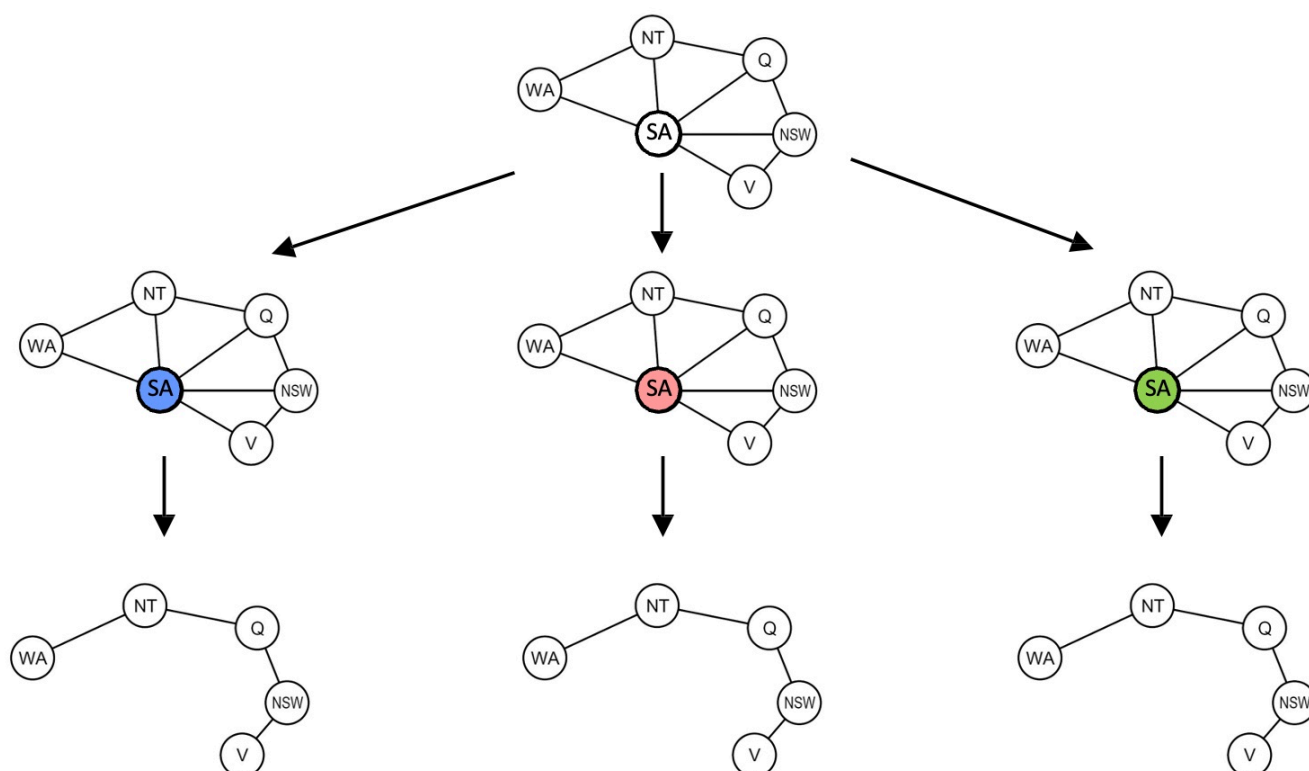
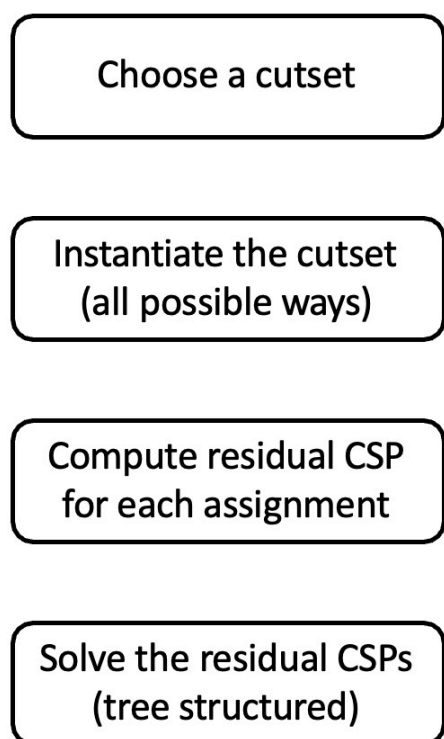
Cut-set conditioning

1. Choose a subset S of CSP's variables such that the constraint graph becomes a tree after removal of S .

2. For each possible assignment to variables in S that satisfy all constraints on S ,

 a. Remove from the domain of remaining variables any values that are inconsistent with assignment for S .

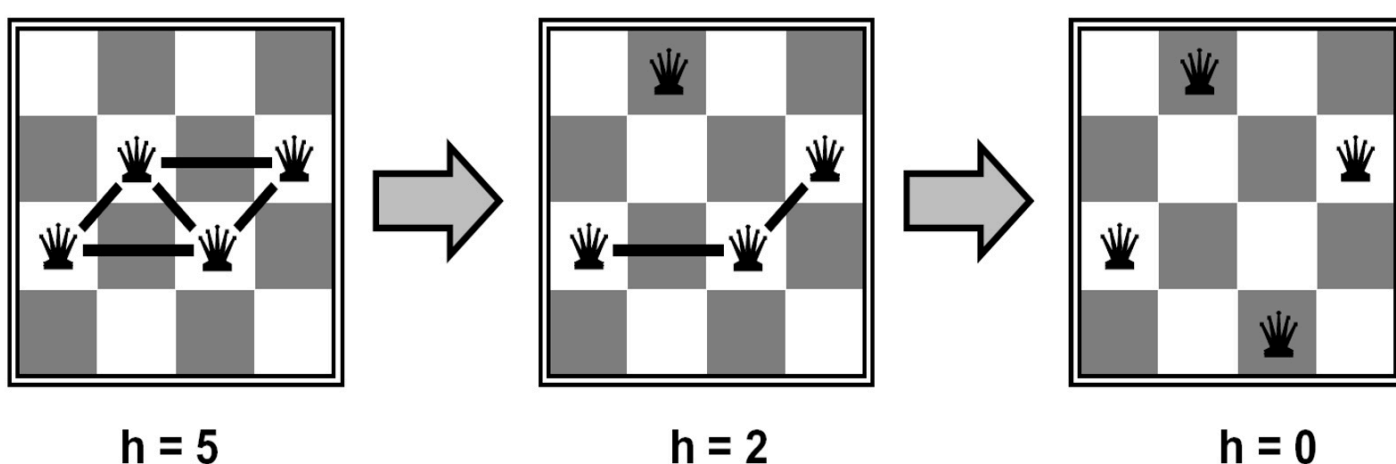
 b. If remaining CSP has solution, return it together with assignment for S .



Iterative Algorithms for CSP:

Local search:

- ↳ Looks only at direct neighbours
- ↳ Variable selection: randomly select any conflicted variable
 - ↳ generate it's children & calculate heuristic
- ↳ Value selection: no. of conflicts as a heuristic
 - ↳ Choose child that violates fewest constraint



$h = 5$

$h = 2$

$h = 0$

- States: 4 queens in 4 columns ($4^4 = 256$ states)
- Operators: move queen in column
- Goal test: no attacks
- Evaluation: $c(n) = \text{number of attacks}$

function MIN-CONFLICTS(*csp*, *max_steps*) **returns** a solution or *failure*

inputs: *csp*, a constraint satisfaction problem

max_steps, the number of steps allowed before giving up

→ only for understanding

current ← an initial complete assignment for *csp*

for *i* = 1 to *max_steps* **do**

if *current* is a solution for *csp* **then return** *current*

var ← a randomly chosen conflicted variable from *csp*.VARIABLES

value ← the value *v* for *var* that minimizes CONFLICTS(*csp*, *var*, *v*, *current*)

 set *var* = *value* in *current*

return *failure*

The MIN-CONFLICTS local search algorithm for CSPs. The initial state may be chosen randomly or by a greedy assignment process that chooses a minimal-conflict value for each variable in turn. The CONFLICTS function counts the number of constraints violated by a particular value, given the rest of the current assignment.

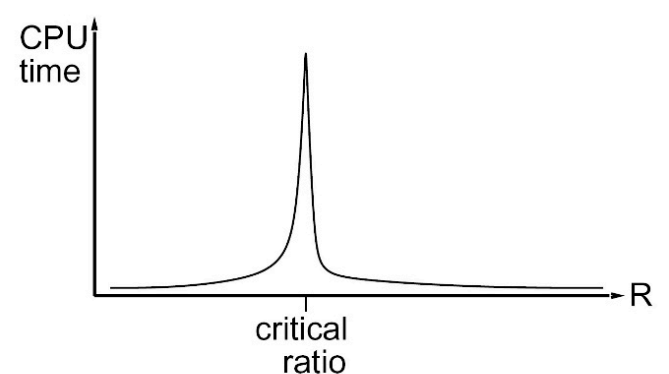
Performance of min-conflicts:

- Given a random initial state, can solve n-queens in almost constant time for arbitrary n with high probability (e.g., $n = 10,000,000$)!
- The same appears to be true for any randomly generated CSP *except* in a narrow range of the ratio.
- However, local search is **both incomplete and suboptimal** and so won't necessarily converge to an optimal solution.
- Additionally, there is a critical ratio around which using local search becomes extremely expensive:

$$R = \frac{\text{number of constraints}}{\text{number of variables}}$$

The time needed by iterative algo to get to solution

- R very small i.e. Number of Constraints is less and variables are a lot
- R very large i.e. Number of Constraints are lot and few variables



Hill Climbing

- Start wherever
- Repeat: move to the best neighboring state
- If no neighbors better than current, quit

function HILL-CLIMBING(*problem*) **returns** a state that is a local maximum

current $\leftarrow$ MAKE-NODE(*problem*.INITIAL-STATE)

loop do

neighbor $\leftarrow$ a highest-valued successor of *current*

if *neighbor*.VALUE $\leq$ *current*.VALUE **then return** *current*.STATE

current $\leftarrow$ *neighbor*

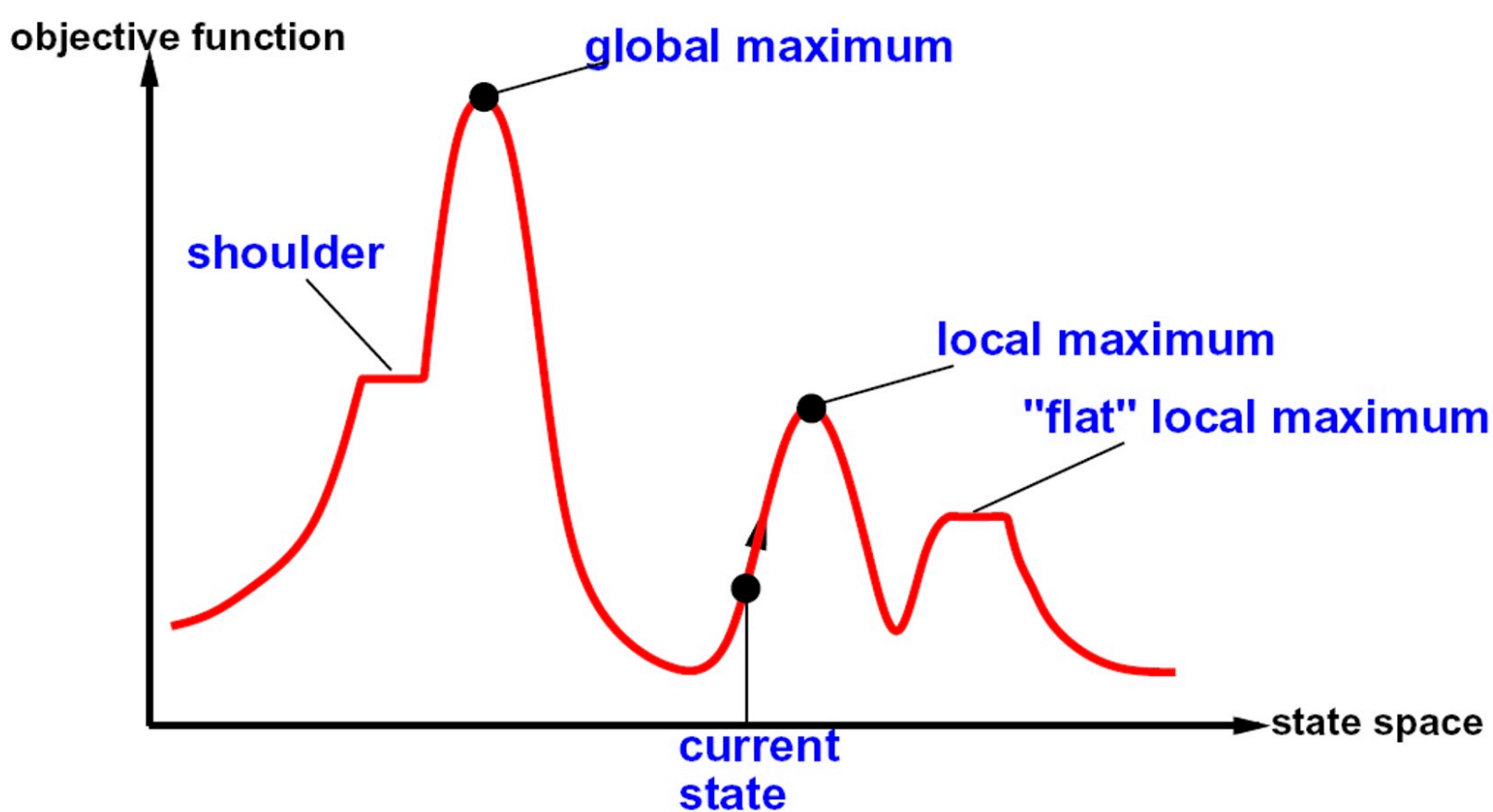
Hill-climbing on 8-Queen:

↳ Randomly generate 1 8-queen starting state

↳ 14% of the time it solves the problem.

↳ 86% of the time gets stuck on local minima/shoulder

↳ If we allow side-ways movement to escape shoulder with a limit, then it goes from 14% to 94%.



Variants of hill climbing

Stochastic hill climbing

At random choose from among uphill moves.

↳ At each step.

↳ choose best neighbours with probability p (greedy)

↳ Choose random neighbours with probability $(1-p)$ (Random)

↳ If $p = 0.8$, then 80% of the time choose best neighbour and 20% time choose random neighbour

First-choice / Greedy hill climbing

↳ Move to the neighbour with largest value

Random Walk:

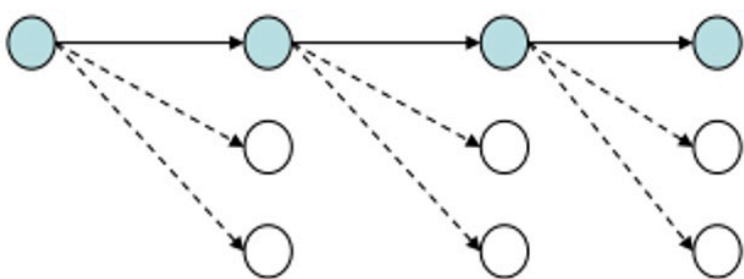
↳ Move to a random neighbour

Random restart hill climbing:

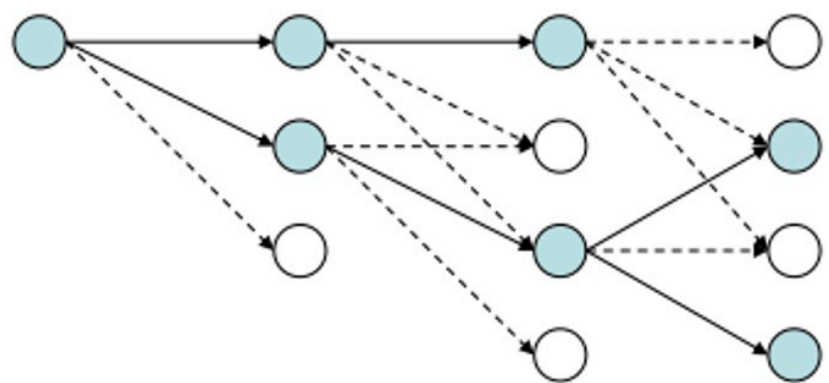
↳ Conducts series of hill-climbing searches from randomly generated initial state until a goal is found

Local beam search:

↳ Instead of selecting best neighbour, select k best neighbours



Greedy Search



Beam Search $k=2$

